AI 388 / CS 388 Natural Language Processing

Module 6: Pre-training and Tokenization

Topics 16, 17

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Full derivations** (added Oct 20): every formula derived step by step with nothing assumed, plus low-dimensional by-hand examples of the algorithms worked in lecture and "conceptual checks" for Viterbi/CKY.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide **1**

16 Encoders: Masked Language Modeling and BERT **3**

16.1 Why not a Transformer LM? Why masking? 3

16.2 BERT pre-training 3

16.3 Applying BERT: fine-tuning 4

16.4 Derivations and worked numbers 4

16.5 Encoder vs. decoder vs. encoder–decoder 5

16.6 Past exam / exercises 6

16.7 Practice 7

17 Subword Tokenization (BPE, WordPiece) **8**

17.1 Byte Pair Encoding (BPE; Sennrich et al. 2016) 8

17.2 WordPiece / unigram LM (SentencePiece) 8

17.3 Derivations 9

17.4 Greedy (longest-match) tokenization, the exam algorithm 9

17.5 Practice 11

TOPIC 16

Encoders: Masked Language Modeling and BERT

Sources: Lectures 6.0.0 BERT: Masked Language Modeling, 6.0.1 BERT: Model and Applications, 6.1.0 Seq2seq Models, 6.1.1 BART, 6.1.2 T5; slides seg-68, seg-69, seg-s2s1–3; Exercises 6.0, 6.1; Fall 2023 Q1(12).

In plain words

To understand a word you want context from *both* sides, but a normal (left-to-right) LM can't look right, or it would see the answer. **Masked language modeling** hides about 15% of the tokens (replaces them with [MASK]) and trains a *bidirectional* Transformer encoder to fill them in from everything else. Nothing to cheat from, since the answer isn't in the input. Pre-train BERT this way on huge text, then **fine-tune** the whole network (with a small classifier on the [CLS] vector) on your task. BERT is great at *analysis* (classification, tagging, sentence pairs) but can't naturally generate text.

16.1 Why not a Transformer LM? Why masking?

- **ELMo** (2018): forward and backward LSTM LMs, *concatenated*. Two independent one-sided views, not deeply merged.
- **GPT**: a one-sided (causal) Transformer LM. It only looks back.
- A bidirectional Transformer trained to predict the next word would **cheat**: position i attends to word $i+1$.
- **MLM fix:** "John visited [MASK] yesterday" $\rightarrow$ predict *Madagascar*. The model sees all other words in both directions but not the target. Only the masked positions incur loss (about 15%). If you masked everything there'd be nothing to predict from.

16.2 BERT pre-training

BERT objective = MLM + next sentence prediction (NSP)

Input: `[CLS] text chunk 1 [SEP] text chunk 2`. Embeddings = word piece + **position** (learned integer embedding) + **segment** (chunk 1 vs. 2).

1. **MLM:** mask 15% of the tokens and predict them (softmax over the vocabulary at those positions). In the BERT paper, the chosen tokens are replaced with [MASK] 80% of the time, a random token 10%, and left unchanged 10%. (The lecture just says "mask out 15%"; the 80/10/10 detail is from Devlin et al. and explains Fall 2023 Q12 option C.)
2. **NSP:** 50% of the time chunk 2 truly follows chunk 1; 50% it's a random chunk ("NotNext"). Predict IsNext/NotNext from the **[CLS]** output vector. Later work (RoBERTa) found NSP isn't important.

Model sizes

Both are Transformer encoders (bidirectional self-attention, no causal mask). **BERT-Base:** 12 layers, $d_{model} = 768$, 12 heads, about 110M parameters. **BERT-Large:** 24 layers, $d_{model} = 1024$, 16 heads, **340M** parameters. Input tokens are **WordPieces** (Topic 17).

16.3 Applying BERT: fine-tuning

Fine-tuning recipe

1. Feed the input (single sentence, or sentence pair joined with [SEP]).
2. **Classification:** take the final [CLS] vector → linear/small FFN → softmax. **Tagging:** classify each token's output vector (e.g. NER tags).
3. Train the new head **and all of BERT** on task data for a few epochs with a **small learning rate** (plus tricks such as warmup). Top layers change most, lower layers little, which preserves the pre-trained knowledge.

- **Fine-tuning helps BERT** (unlike ELMo, where fine-tuning vs. frozen features hurt). BERT is rarely used as frozen embeddings.
- **Sentence-pair tasks** (paraphrase, entailment) improved most. Self-attention aligns the two sentences. Most of this is learned during fine-tuning, not from NSP.
- **GLUE benchmark:** BERT-Large beat task-specific state-of-the-art models and ELMo/GPT by large margins.
- **Analysis** (Clark et al.): some heads attend to the previous/next token; some attend broadly to [SEP]/periods; some track syntax (verbs → direct objects, nouns → modifiers), learned without supervision.
- **RoBERTa:** about 10× more data, **dynamic masking** (different masks each epoch), drops NSP. Also **whole-word masking** (mask all pieces of a word).
- **Can't generate well:** you'd append [MASK] and refill repeatedly, which is $O(n^2)$ over the output length, and parallel fills may not be coherent. BERT is meant for **analysis** tasks.

16.4 Derivations and worked numbers

A. The MLM loss, written out

1. Corrupt the input $\mathbf{x}$ into $\tilde{\mathbf{x}}$ by choosing a set M of positions (about 15%).
2. Run the bidirectional encoder on $\tilde{\mathbf{x}}$. At each $i \in M$: $P(\cdot \mid \tilde{\mathbf{x}}) = \text{softmax}(W\mathbf{h}_i)$ over the vocabulary.
3. Loss $= \sum_{i \in M} -\log P(x_i \mid \tilde{\mathbf{x}})$. Positions not in M contribute nothing.
4. Contrast with the causal LM loss $\sum_{i=1}^{n} -\log P(x_i \mid x_{<i})$: every position, but only left context. MLM: both-side context, about 15% of positions. So BERT gets fewer training signals per sequence.
5. Expected counts for $n = 200$: $|M| = 30$. With 80/10/10: 24 [MASK], 3 random tokens, 3 unchanged (all 30 still predicted).

Worked example: MLM loss at two masked positions

"the [MASK] sat on the [MASK]", true words *cat* (pos 2) and *mat* (pos 6). Suppose the model gives $P(\text{cat}) = 0.5$ at pos 2 and $P(\text{mat}) = 0.2$ at pos 6. Loss $= -\ln 0.5 - \ln 0.2 = 0.6931 + 1.6094 = 2.3026$ (mean over masked positions: 1.1513). Nothing is added for the four unmasked words.

B. Where BERT's parameter counts come from

1. Per layer (no biases/LN): attention $4d^2$ + FFN $2d \cdot 4d = 8d^2$, so $12d^2$.
2. BERT-Base, $d = 768$, 12 layers: $12 \times 12 \times 768^2 = 84.9$M. WordPiece embeddings $30{,}522 \times 768 = 23.4$M; positions $512 \times 768 = 0.4$M; segments 2×768. Total $\approx$ 108.8M, plus biases and LayerNorms $\approx$ 110M. ✓
3. BERT-Large, $d = 1024$, 24 layers: $24 \times 12 \times 1024^2 = 302$M, plus embeddings $\approx$ 32M, so $\approx$ 334M, plus biases/LN $\approx$ 340M. ✓
4. Most parameters are in the layers, and within each layer two thirds are in the FFN.

C. Why BERT generation is $O(n^2)$ (lecture claim)

1. To add token k, append [MASK] and run the full bidirectional encoder over all $\approx k$ tokens. Since attention is bidirectional, earlier positions' outputs change when a token is added, so nothing can be reused.
2. Generating m tokens costs $\sum_{k=1}^{m} O(k) = O(m^2)$ position-computations (each of which itself contains attention over k tokens).
3. A causal decoder can keep each earlier position's keys and values, because adding a future token doesn't change them; each new token only computes its own row.

D. The seq2seq factorization (lecture 6.1.0)

1. Chain rule, conditioned on $\mathbf{x}$: $P(\mathbf{y} \mid \mathbf{x}) = \prod_{i=1}^{|\mathbf{y}|} P(y_i \mid \mathbf{x}, y_{<i})$, ending when STOP is produced.
2. Training loss: $\sum_i -\log P(y_i^* \mid \mathbf{x}, y_{<i}^*)$ with the gold prefix fed in, exactly like the LM loss but with an extra input.
3. Inference: encoder once on $\mathbf{x}$; then the decoder one token at a time (greedy, beam, or sampling; Topic 18).
4. Each decoder layer has *two* attentions: causal self-attention over $y_{<i}$, then cross-attention whose queries come from the decoder and whose keys/values come from the encoder outputs.

16.5 Encoder vs. decoder vs. encoder–decoder

	attention	pre-training	best for
Encoder (BERT, RoBERTa)	bidirectional	MLM (+NSP)	classification, tagging, sentence pairs
Decoder (GPT, A3 LM)	causal (past only)	next-token LM	generation
Enc–dec (Transformer, BART, T5)	encoder bidirectional; decoder causal + **cross-attention** to the encoder	denoising (BART: reconstruct full text; T5: predict only masked spans)	seq2seq: translation, summarization, QA

- **Seq2seq:** $P(\mathbf{y}|\mathbf{x}) = \prod_i P(y_i \mid \mathbf{x}, y_{1:i-1})$. The encoder runs **once**; the decoder generates left to right until a STOP token, feeding back each output. Two models with separate parameters (and possibly vocabularies). Lower-entropy than language modeling (MT is "difficult, not impossible").
- **BART** (Lewis et al., Meta): corrupt the input (mask spans, delete, permute, ...), encode it, and the decoder outputs the **entire clean text**. A superset of BERT (the encoder alone can be used like BERT). Strong at **summarization**.
- **T5** (Google): mask spans, predict **only the masked spans**. C4 dataset: more *unique* data helps (repeating less data is worse). Multitask/unified text-to-text, e.g. UnifiedQA.

16.6 Past exam / exercises

Midterm Fall 2023 Q1(12)

What would most likely improve fine-tuned accuracy of a pre-trained BERT? A. Masking more than 15% of tokens; B. only left-to-right encoding; C. replacing 100% of masked tokens with random words.

Official answer: A. B removes bidirectionality (BERT's key advantage). C makes inputs noisy and inconsistent: the model can never trust any token and never sees [MASK]. (A is the official answer; later work found somewhat higher masking rates can help.)

Canvas Exercise 6.0

Q1 BERT:

Answer: masks 15% of tokens and predicts them; includes an NSP classification loss. It is *not* trained to generate 100% of tokens, there's no attention-penalty loss, and no grammaticality/noising loss (that's BART). Q2 RoBERTa:

Answer: trains on more data; changes the masks every epoch.

Canvas Exercise 6.1 (seq2seq)

Q1 true:

Answer: runtime quadratic in output length; linear in decoder vocabulary size if layers/lengths are fixed; any classifier network can model $P(y_i|\mathbf{x}, y_{<i})$. Not "decode out of order". Q2 why Transformers:

Answer: reuse computation across decoding steps; pass information over long distances better than RNNs; the decoder can use the whole output prefix. Multiple decoder steps *can't* run in parallel at inference.

Traps

- MLM loss is only on masked positions. Next-token LM loss is on every position.
- BERT is bidirectional (no causal mask); GPT/A3 LM is causal.
- NSP uses [CLS]; MLM uses the masked positions' outputs.
- "BERT is a language model" in the GPT sense is false. It's a *masked* LM and doesn't define $P(\text{sentence})$ left to right.
- BART reconstructs the full sequence; T5 outputs only the masked spans.

16.7 Practice

Practice

1. A 200-token BERT input with 15% masking: about how many positions contribute MLM loss? With a GPT-style LM?
2. Why can't you train BERT's bidirectional encoder with a next-token objective?
3. Which model for: (a) sentiment classification, (b) news summarization, (c) story continuation?

Answers

1. About 30; GPT: 200 (every position predicts its next token). 2. With bidirectional attention, each position can attend to the next token and copy it, a trivial solution. 3. (a) BERT/encoder with [CLS] head; (b) BART/T5 (enc–dec); (c) decoder LM (GPT) with nucleus sampling.

TOPIC 17

Subword Tokenization (BPE, WordPiece)

Sources: Lecture 6.2.0 Word Piece and Byte Pair Encoding; 3.1.1 Other Word Embedding Methods (fastText/subwords); slides seg-subword-segmentation; Exercise 6.2; Fall 2023 Q2(c–e), Q1(15).

In plain words

Word vocabularies are either too small (lots of UNKs, copying needed) or too big (slow softmax). Characters are too fine (sequences become very long). **Subwords** are the compromise: frequent words stay whole, and rare words split into meaningful pieces (*re_ fueling*, *un_ princi_ pled*). Any word can be represented, so there are no UNKs, and related words share pieces (*dog*, *dog s*).

17.1 Byte Pair Encoding (BPE; Sennrich et al. 2016)

Learning BPE merges

1. Start with every character (byte) as its own symbol (often with an end-of-word marker _). Word counts come from a large corpus.
2. Count adjacent symbol pairs, weighted by word frequency.
3. **Merge the most frequent pair** into a new symbol; add it to the vocabulary.
4. Repeat for a fixed number of merges. 30k merges gives a vocabulary of about 30,000 pieces, including many whole words.

To segment new text, apply the learned merges in order (or greedy longest match against the vocabulary, as the exam does).

Worked example: BPE on {low:5, lower:2, newest:6, widest:3} (end marker _)

1. Pair counts: (e,s)=9, (s,t)=9, (t,_)=9, (w,e)=8, … Tie: take the first, **es**.
2. (es,t)=9, so merge **est**. 3. (est,_)=9, so merge **est_**.
3. (l,o)=7, so **lo**. 5. (lo,w)=7, so **low**. 6. (n,e)=6 (tie with (e,w), (w,est_)), so **ne**.

Now “newest” = ne w est_, “lowest” (unseen) = low est_. Ties must be broken by a stated rule.

17.2 WordPiece / unigram LM (SentencePiece)

- Choose pieces that best explain the corpus under a language model: merge (or keep) the pieces that most improve LM likelihood/perplexity, until the target vocabulary size is reached. Google’s SentencePiece library implements it (Google Translate). Lecture: “unigram LM”. BERT uses WordPieces.
- **Bostrom & Durrett (2020):** BPE produces less linguistically plausible units. Unigram LM gives morphological splits (*tri cycle s*) and some evidence of better pre-trained models.
- Ensembling across multiple tokenizations is also explored.

17.3 Derivations

A. The BPE example, checked merge by merge (verified by script)

Corpus {low:5, lower:2, newest:6, widest:3}, end marker _; ties go to the pair that appears first.

merge	pair	count	tied pairs (count)
1	e+s	9	(s,t), (t,_)
2	es+t	9	(t,_)
3	est+_	9	–
4	l+o	7	(o,w)
5	lo+w	7	–
6	n+e	6	(e,w), (w,est_)
7	ne+w	6	(w,est_)
8	new+est_	6	–

Where the 9 in merge 1 comes from: "es" occurs once in *newest* (6 copies) and once in *widest* (3 copies), so $6 + 3 = 9$. After 8 merges: *low* $\rightarrow$ low _, *lower* $\rightarrow$ low e r _, *newest* $\rightarrow$ newest_, *widest* $\rightarrow$ w i d est_.

B. Unigram-LM segmentation (the WordPiece/SentencePiece idea), on a tiny example

1. Give each piece a probability; a segmentation's probability is the product of its pieces' probabilities. Choose the most probable segmentation.
2. Pieces: tri 0.05, cycle 0.04, s 0.10, tricycle 0.001, t 0.05, ri 0.01, cy 0.02, cle 0.01.
3. "tricycles": tri·cycle·s $= 0.05 \cdot 0.04 \cdot 0.10 = 2 \times 10^{-4}$; tricycle·s $= 0.001 \cdot 0.10 = 10^{-4}$; t·ri·cy·cle·s $= 0.05 \cdot 0.01 \cdot 0.02 \cdot 0.01 \cdot 0.1 = 10^{-8}$.
4. Best: **tri cycle s**, the morphological split the lecture shows. Because every factor is < 1, more pieces usually cost probability, but frequent pieces can still beat one rare long piece.
5. Training chooses the piece inventory (and probabilities) that maximize the corpus likelihood, i.e. minimize perplexity. BPE instead adds pieces by raw pair frequency.

C. Why sequences can't get shorter than word tokenization

Every vocabulary piece lies inside one word (pieces don't cross word boundaries), so each word becomes ≥ 1 piece. Total pieces $\geq$ number of words, with equality only when every word is a single piece.

17.4 Greedy (longest-match) tokenization, the exam algorithm

Greedy segmentation

At each position, take the **longest** vocabulary piece that matches starting there. Emit it and continue after it. (If nothing matches, fall back to a single character/UNK.) Greedy is *not* guaranteed to use the fewest pieces.

Canvas Exercise 6.2

Q1 advantages:

Answer: any word can be represented; a plural and its singular might differ only in the token "s". Not "sequences get shorter" (they stay the same or get longer); not "misspellings get the same representation" (no guarantee).
Q2 vocabulary {A, AA, AAB, ABB, BAAA}, greedy on AABAAA:

Answer: AAB AA A (no AABA…; then AAA isn't in the vocabulary but AA is; then A).
Q3 fewest pieces:

Answer: AA BAAA (2 pieces vs. greedy's 3).

Midterm Fall 2023 Q2(c)–(e)

Subword vocabulary {good, x, q, not, gre, at, great} with greedy tokenization. Data: `goodx+`, `great+`, `not greatq−`, `not good−`.
(c) Segmentations: goodx → good x; great → great; greatq → great q; not → not. New BoW vocabulary = the 7 subwords in order (good, x, q, not, gre, at, great). Features:

Official answer: (1,1,0,0,0,0,0), (0,0,0,0,0,0,1), (0,0,1,1,0,0,1), (1,0,0,1,0,0,0). (The question meant features for each example; partial credit if computed for the vocabulary.)
(d) `greqat` →

Official answer: gre q at ("great" doesn't match because of the q).
(e) Typos anywhere in a word: subword tokenization vs. edit-distance repair?

Official answer: Typo repair is generally less fragile. Subwords split typo'd words oddly (gre q at loses "great"), while repair maps it back.

Midterm Fall 2023 Q1(15)

Train on singular nouns, test on plurals:

Official answer: stemming, subword tokenization, pre-trained embeddings help. dog/dogs share the piece *dog*.

Traps

- Greedy = longest match from the left, piece by piece. Don't search for the globally fewest pieces unless asked.
- Subword tokenization never makes sequences shorter than word tokenization.
- BPE merges are learned by *frequency*; WordPiece/unigram LM by *likelihood*.
- After segmenting, a BoW vocabulary over subwords counts *pieces*. Use the vocabulary order given.
- The underscore marks a word boundary (start of a word in the lecture's notation, *_eco tax*); continuation pieces have no underscore.

17.5 Practice

Practice

1. Vocabulary {un, u, n, h, a, p, py, happy, ha, ppy}. Greedy-tokenize “unhappy” and “uhappy”.
2. Continue the BPE example: what is merge 7 (ties broken by first occurrence in the order low, lower, newest, widest)?
3. Give one reason subwords help a sentiment model on A2’s typo dev set, and one reason they might not.

Answers

1. un happy; u happy.
2. After merge 6: low_ (5), low e r _ (2), ne w est_ (6), w i d est_ (3). Pair counts: (low,_)=5, (low,e)=2, (e,r)=2, (r,_)=2, **(ne,w)=6**, **(w,est_)=6**, (w,i)=3, (i,d)=3, (d,est_)=3. Tie at 6; (ne,w) occurs first, so merge 7 is **new** (and merge 8 is newest_).
3. Helps: a typo’d word still shares most pieces with the correct word (A2 typos never touch the first 3 characters). Might not: a typo can change the segmentation entirely, giving pieces with unrelated embeddings.